Modul Media Programming, Lehrveranstaltung AV Programming



Prof. Dr. Eva Wilk
HAW Hamburg, Fakultät INF, Studiengang Medieninformatik

Thema 7 - Übersicht

Audio- und Video-Inhaltsanalyse: führt viele der bisherigen Inhalte zusammen.

Leitfrage: Wie werden aus Audio- und Videodaten inhaltlich nutzbare Hinweise?

Ziel: aus Daten Hinweise auf Ereignisse, Struktur oder Bedeutung gewinnen.

Anwendungen: automatische Segmentierung von Podcasts, Tempo- und Beat-Erkennung, Audio-Fingerprinting, Shot-Erkennung in Videos, Objekt- oder Aktivitätserkennung.

Ergebnisse: Technische, interpretierbare Merkmalen ... ML-gestützte Inhaltsanalyse

Thema 7 - Lernziel

Die Studierenden analysieren Audio- und Videodaten anhand ausgewählter Merkmale und einfacher ML-gestützter Verfahren mit Python-Werkzeugen wie librosa und OpenCV, um inhaltsbezogene Informationen zu bestimmen, Verfahren und Ergebnisse fachlich einzuordnen sowie deren Aussagekraft und Grenzen in medieninformatischen Anwendungskontexten zu beurteilen.

Audio- und Video-Inhaltsanalyse

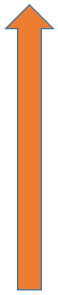
Drei Ebenen:

1. Merkmalsextraktion
2. Inferenz / Entscheidung
3. ML-gestützte Inhaltsanalyse

Anwendungen:

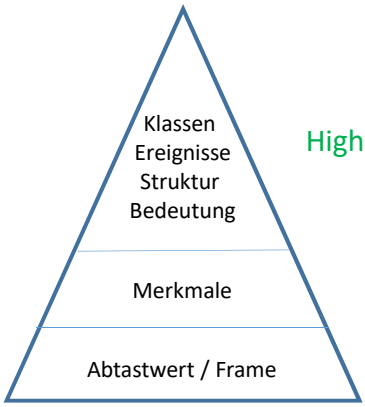
- Automatische Segmentierung von Podcasts
- Tempo- und Beat-Erkennung
- Audio-Fingerprinting
- Shot-Erkennung in Videos
- Objekt- oder Aktivitätserkennung.

Grundmodell der Inhaltsanalyse



- Inferenz: verbindet Merkmale mit Deutung →
- Merkmale: beschreiben Struktur, nicht Bedeutung →

Rohsignal - noch kein Inhalt →



High-Level-Features

Mid-Level-Features

Low-Level-Features

Grundmodell der AV-Inhaltsanalyse

- **Low-Level-Features** (eher transparent, wenig inhaltliche Aussage)
  - aus Abtastwerten: RMS, Spektrum, spektrale Differenz
  - aus Frames: Helligkeit, Farbe, Frame-Differenz
- **Mid-Level-Features** (mittlere Transparenz, mittlere inhaltliche Aussage)
  - Audio: Pitch-Verlauf, Beat, Segmentgrenzen
  - Video: Shots, Keyframes, Tracking-Spuren, Bewegungsmuster
- **High-Level-Inferenz** (häufig schwer nachvollziehbar, starke inhaltliche Aussage)
  - “neuer Abschnitt”
  - “vermutlich Musik”
  - “Shot-Wechsel”
  - “Highlight”

Grundmodell der Inhaltsanalyse

- Ein Rohsignal ist noch kein Inhalt.
- Ein Merkmal ist zunächst nur eine beschreibende Größe, etwa Energie, spektrale Lage oder Bildänderung.
- Durch Inferenz entsteht daraus eine inhaltliche Aussage. Ein Peak in einer RMS-Kurve bedeutet zunächst nur: Hier steigt die Signalenergie. Das könnte eine Aktivitätsphase sein, aber man kann noch nicht direkt auf ein ganz bestimmtes Ereignis schließen.

Unterscheidung: Signal, Merkmal, Inferenz.
Machine Learning: Einsatz, wenn solche Zuordnungen nicht mehr gut mit wenigen expliziten Regeln gelingen, sondern aus Trainingsdaten gelernt werden

Klassische Merkmale sind oft transparenter und leichter zu erklären.
ML ist oft leistungsfähiger, aber datenabhängig und weniger interpretierbar.

Thema 7 – Anwendungen von AV-Inhaltsanalyse

1. Automatische Segmentierung von Podcasts
2. Tempo- und Beat-Erkennung
3. Audio-Fingerprinting
4. Shot-Erkennung in Videos
5. Objekt- oder Aktivitätserkennung

1. Automatische Segmentierung von Podcasts

Eine Podcast-Folge wird in inhaltliche Abschnitte mit Marken und Titeln unterteilt

Beispiel:

- 00:00 Begrüßung
- 02:15 Nachrichtenüberblick
- 10:40 Interview mit Gast
- 28:30 Hörerfragen
- 35:00 Fazit

Automatisch Hinweise finden durch:

- längere Pausen
- Musiktrenner
- Sprecherwechsel
- starke Änderungen in Energie oder Spektrum
- wiederkehrende Formatelemente

2. Tempo- und Beat-Erkennung

Beat-Erkennung:
Es werden die regelmäßigen Zählzeiten bzw. Pulse im Signal gefunden

Tempo-Erkennung:
Aus diesen regelmäßigen Pulsen wird die Geschwindigkeit der Musik bestimmt, meist in BPM („beats per minute“)

→ Übergang von Merkmalen zu Inferenz: Aus Signalveränderungen wird auf eine musikalische Zeitstruktur geschlossen

3. Audio-Fingerprinting

-> Wiedererkennung von Audiosignalen anhand eines kompakten, charakteristischen Merkmalsprofils

-> bezeichnet Verfahren zur Identifikation oder zum Abgleich von Audiosignalen auf Basis kompakter, robust extrahierter Merkmalsrepräsentationen.

- beantwortet die Frage: „Ist dieses Signal identisch oder ähnlich zu einem bereits bekannten Audioeintrag?“

Eigenschaften: kompakt, charakteristisch, robust gegenüber Störungen:

→ Verfahren soll auch dann noch funktionieren, wenn

- Umgebungsgeräusche vorhanden sind,
- das Signal komprimiert wurde,
- nur ein kurzer Ausschnitt vorliegt,

} kompakt

- die Aufnahme über Mikrofon statt direkt aus der Datei stammt.

4. Shot-Erkennung in Videos

-> bezeichnet Verfahren zur automatischen Identifikation von Shot-Grenzen, also Übergängen zwischen zusammenhängenden Kameraeinstellungen in Videosequenzen.

-> Beispiel: Ein Nachrichtenbeitrag enthält

- Totale des Studios
- Schnitt auf Moderatorin
- Schnitt auf Einspielfilm
- Schnitt auf Interviewausschnitt

Shot-Erkennung wird eingesetzt für:

- Video-Segmentierung
- Browsing und Navigation
- *Keyframe*-Auswahl (*Bild, das stellvertretend für Shot, Szene oder Video steht*)
- Archivierung
- Inhaltsanalyse
- *Highlight*-Erkennung (*Segment, das inhaltlich besonders relevant ist → inhaltliche Bewertung*)

5. Objekt- oder Aktivitätserkennung.

Objekterkennung: automatische Identifikation und ggf. Lokalisierung von Objekten in Bild- oder Videodaten.

Beispiele: Person, Auto, Ball, Hund, Fahrrad

Aktivitätserkennung: automatische Erkennung von Handlungen oder zeitlichen Ereignissen in Videodaten.

Beispiele: eine Person läuft, jemand hebt die Hand, ein Ball wird geworfen, zwei Personen sprechen miteinander, ein Auto biegt ab

Audio-Inhaltsanalyse

Typische Aufgaben der Audio-Inhaltsanalyse sind zum Beispiel:

- Grundfrequenzerkennung, Beat- und Tempo-Erkennung, Musiktranskription, musikalische Strukturerkennung, Fingerprinting oder semantische Aufgaben wie Genre- oder Emotionserkennung.

Nicht alle diese Aufgaben liegen auf demselben Niveau.

- relativ signalnah: Pitch, Beat.
- stark semantisch: Genre, Emotion → deutlich interpretations- und datenabhängiger.

Daher: Trennung zwischen Merkmalsextraktion und Inferenz

Termin 7: zunächst transparente und direkt nachvollziehbaren Bausteine: Energie, Spektrum, Pitch und Beat.

Audio-Inhaltsanalyse

Typische Analyse-Fragen: was soll im Audiomaterial erkannt werden

- Wo liegen Aktivitätsphasen, Ereignisse oder Pausen?
- Welche zeitlichen und spektralen Eigenschaften liegen vor?
- Welche Grundfrequenz- oder Tonhöhenstruktur liegt vor?
- Welche Puls-, Beat- oder Tempostruktur liegt vor?
- Welche wiederkehrenden Muster oder formalen Abschnitte sind erkennbar?
- Welche Zuordnungen, Wiedererkennungen oder Klassifikationen sind naheliegend?

Beispiele für Analyseziele:
Segmentierung, Pitch-/F0-Schätzung, Beat-/Temposchätzung, Fingerprinting, Klassifikation

Audio-Inhaltsanalyse

Mit welchen Merkmalen und Verfahren kann man diese Fragen beantworten:

- Aktivität, Ereignisse, Pausen → z. B. RMS / Energie, Hüllkurve, Hinweise auf Ereignisbeginne (Onsets)
- zeitliche und spektrale Eigenschaften → z. B. Spektrum, Spektrogramm, spektraler Schwerpunkt
- Tonhöhen- und Grundfrequenzstruktur → z. B. F0-Schätzung, Pitch-Verlauf
- Puls-, Beat- und Tempostruktur → z. B. Onset-Stärke, Beat-Tracking, Tempo-Schätzung
- wiederkehrende Muster und formale Abschnitte → z. B. Segmentierung, Ähnlichkeitsmaße, Strukturmerkmale
- Zuordnung, Wiedererkennung, Klassifikation → z. B. Fingerprints, Merkmalskombinationen, Embeddings, gelernte Repräsentationen

Onset

Onset
Beginn von Klang oder Ereignis

Beispiele: Anschlag Klavierton, Trommelschlag, Beginn Silbe, Einsetzen neuer Ton

Onset-Hinweis (Hinweis auf ein Einsetzen von Klang oder Ereignis)
Anzeichen im Signal, das ein Einsetzen stattfinden könnte

Beispiele: plötzlicher Anstieg der Energie, abrupte spektrale Veränderung, deutliche Änderung im Zeitverlauf

Onset-Stärke (Stärke des Hinweises)
Wie stark zeigt Signal zu einem bestimmten Zeitpunkt Merkmale eines Einsetzens

Wird frameweise berechnet → Kurve über Zeit → Peaks in dieser Kurve = mögliche Onsets

Audio-Inhaltsanalyse

`librosa` als Werkzeug für die Audioanalyse

- Python-Bibliothek für Audio- und Musiksignalverarbeitung
- geeignet für:
  - Laden und Vorverarbeiten von Audiodaten
  - Berechnung typischer Audio-Merkmale
  - Pitch-/F0-Schätzung
  - Onset-, Beat- und Tempoanalyse
- Vorteil:
  - kompakte Funktionen
  - schnelle Prototypen
  - gute Verbindung von Signal, Merkmal und Verfahren

Beispiel 25: RMS und spektraler Schwerpunkt

Beispielmerkmale:

- RMS / Energie
- spektraler Schwerpunkt

Analysefragen dazu:

- Wo liegt Aktivität im Signal vor?
- Wo verändert sich die spektrale Verteilung?
- Typische Nutzung:
- grobe Segmenthinweise
- Vergleich von Signalabschnitten
- erster Zugang zur Inhaltsstruktur

Spektraler Schwerpunkt

- beschreibt die mittlere spektrale Lage der Energie eines Signals
- Maß für die spektrale Helligkeit (nicht für die Tonhöhe)
- Verlauf: zeigt Veränderungen der Klangcharakteristik – Beispiel: dunklere, tonale und hellere, rauschhafte Signalanteile
- engl.: spectral centroid, spectral center of gravity
- librosa: `spectral_centroid`

Spektraler Schwerpunkt

Indikator für wahrgenommene Helligkeit von Klängen:

$$SC = \frac{\sum_{k=0}^{N-1} |X[k]| \cdot f[k]}{\sum_{k=0}^{N-1} |X[k]|}$$

Summe über alle Werte |X[k]|

k: Frequenzstützpunkte

f: Frequenz

|X[k]| : Betragsspektrum an dieser Stelle des Spektrums

Beispiel 25: RMS und spektraler Schwerpunkt

```
import matplotlib.pyplot as plt
import librosa

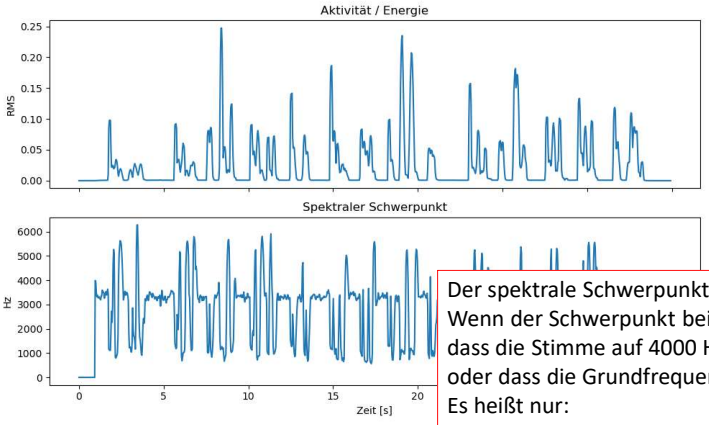
y, sr = librosa.load("audio_Ottos_Mops.wav", sr=None, mono=True)
rms = librosa.feature.rms(y=y)[0]
centroid = librosa.feature.spectral_centroid(y=y, sr=sr)[0]
times = librosa.times_like(rms, sr=sr)

fig, ax = plt.subplots(2, 1, figsize=(10, 6), sharex=True)
ax[0].plot(times, rms)
ax[0].set_ylabel("RMS")
ax[0].set_title("Aktivität / Energie")

ax[1].plot(times, centroid)
ax[1].set_ylabel("Hz")
ax[1].set_xlabel("Zeit [s]")
ax[1].set_title("Spektraler Schwerpunkt")

plt.tight_layout()
plt.show()
```

Beispiel 25: RMS und spektraler Schwerpunkt



Der spektrale Schwerpunkt ist kein Pitch-Maß.
Wenn der Schwerpunkt bei 4000 Hz liegt, heißt das nicht:
dass die Stimme auf 4000 Hz „singt“
oder dass die Grundfrequenz 4000 Hz ist
Es heißt nur:
dass die spektrische Energie im Mittel in diesem Bereich gewichtet ist

Aufgabe 14 – Interpretation von Merkmalen

Vergleichen Sie zwei Audiodateien, z. B. Sprache vs. Musik oder Sprache vs. Geräusch.

Berechnen Sie RMS und spektralen Schwerpunkt.

Formulieren Sie anschließend zwei Annahmen, wie sich beide Signale unterscheiden, und prüfen Sie diese an den Kurven.

Beispiel 25: RMS und spektraler Schwerpunkt

```
import matplotlib.pyplot as plt
import librosa

y, sr = librosa.load("audio_Ottos_Mops.wav", sr=None, mono=True)
rms = librosa.feature.rms(y=y)[0]
centroid = librosa.feature.spectral_centroid(y=y, sr=sr)[0]
times = librosa.times_like(rms, sr=sr)

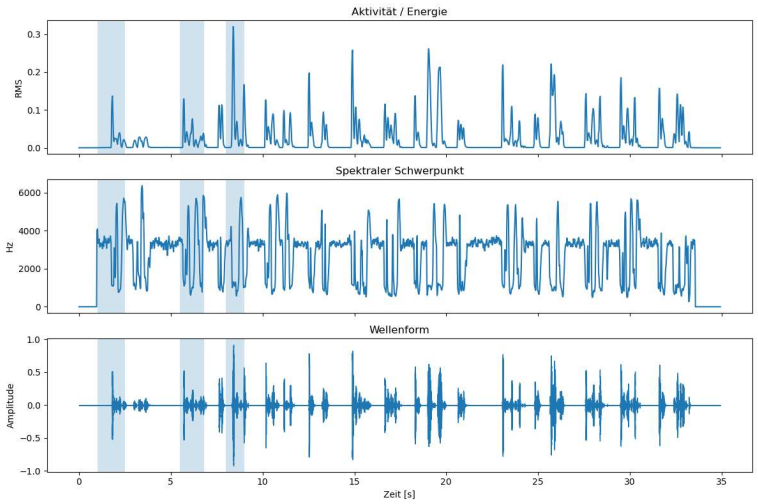
fig, ax = plt.subplots(2, 1, figsize=(10, 6), sharex=True)
ax[0].plot(times, rms)
ax[0].set_ylabel("RMS")
ax[0].set_title("Aktivität / Energie")

ax[1].plot(times, centroid)
ax[1].set_ylabel("Hz")
ax[1].set_xlabel("Zeit [s]")
ax[1].set_title("Spektraler Schwerpunkt")

plt.tight_layout()
plt.show()
```

Ergänzen Sie den Code so, dass
a. als drittes Diagramm die Wellenform des Signals dargestellt wird,
b. die Wellenform in das RMS-Diagramm eingezeichnet wird.

Beispiel 25: RMS und spektraler Schwerpunkt



Beispiel 26: Grundfrequenz schätzen

Übergang von signal-nahen Merkmalen zu strukturbezogener Inferenz

Analysefrage:

- Welche Tonhöhenstruktur liegt vor?

Verfahren:

- Schätzung Grundfrequenz (F=0), Pitch-Verlauf

Audio-Inhaltsanalyse - Tonhöhenstruktur

`librosa.yin`

- schätzt die Grundfrequenz (F0) frameweise.
- Pro Frame wird untersucht, bei welcher Verzögerung sich das Signal periodisch wiederholt.
- Die gefundene Periodendauer wird anschließend in Hertz umgerechnet.

```
f0 = librosa.yin(
    y,
    fmin=librosa.note_to_hz("C2"),
    fmax=librosa.note_to_hz("C7"),
    sr=sr )
```

Beispiel 26: Grundfrequenz schätzen

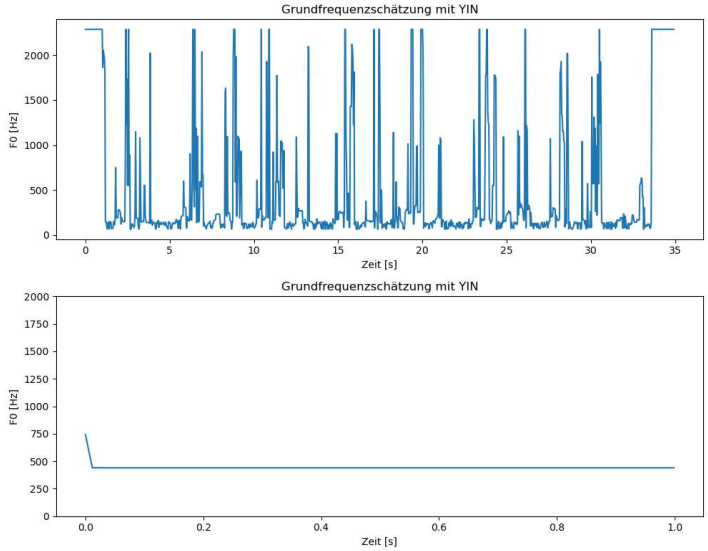
```
y, sr = librosa.load("audio_Ottos_Mops.wav", sr=None, mono=True)
#y, sr = librosa.load("sinus_440Hz.wav",sr=None, mono=True)

f0 = librosa.yin(
    y,
    fmin=librosa.note_to_hz("C2"),
    fmax=librosa.note_to_hz("C7"),
    sr=sr
)

times = librosa.times_like(f0, sr=sr)

plt.figure(figsize=(10, 4))
plt.plot(times, f0)
plt.xlabel("Zeit [s]")
plt.ylabel("F0 [Hz]")
plt.title("Grundfrequenzschätzung mit YIN")
plt.tight_layout()
plt.show()
```

Beispiel 26: Grundfrequenz schätzen



Audio-Inhaltsanalyse – Beats erkennen

Analysefrage:

- Welche Puls-, Beat-, Tempostruktur liegt vor?

Verfahren:

- Onset-Stärke
- Beat-Tracking
- Tempo-Schätzung

Beispiel 27: Beat bzw. Tempo schätzen

`librosa.beat.beat_track`

- schätzt das globale Tempo in BPM (Beats per Minute)
- basiert auf Onset-Struktur des Signals
- schätzt daraus `tempo` und `beat`-Zeitpunkte

```
f0 = librosa.yin(
    y,
    fmin=librosa.note_to_hz("C2"),
    fmax=librosa.note_to_hz("C7"),
    sr=sr )
```

Beispiel 27: Beat bzw. Tempo schätzen

```
import librosa

y, sr = librosa.load("Waltz_for_Ruth.wav", sr=None, mono=True)

tempo, beats = librosa.beat.beat_track(y=y, sr=sr, units="time")

print("Geschätztes Tempo (BPM):", tempo)
print("Beat-Zeitpunkte (s):", beats[:20])
```

Beispiel 27: Beat bzw. Tempo schätzen

```
Geschätztes Tempo (BPM): 107.67
Beat-Zeitpunkte (s): ['17.53', '18.08', '18.62', '19.17', '19.71',
'20.25', '20.78', '21.32', '21.97', '22.56', '23.16', '23.75',
'24.33', '24.96', '25.58', '26.17', '26.77', '27.35', '27.95',
'28.48']
```

Aufgabe 15 – Metronom mit erkanntem Tempo zu Musik abspielen

Prüfen Sie das Ergebnis von Beispiel-Code 27.

Erstellen Sie dazu, passend zum Ergebnis aus Beispiel-Code 27, eine Metronomspur.

Spielen Sie die Metronomspur und die Musik gleichzeitig ab.

Aufgabe 16 – Anwendung und Verfahren zuordnen

Ordnen Sie den Anwendungen das passendere Verfahren zu und begründen Sie kurz:

a) Beat-Erkennung für Musik-App

b) Grundfrequenzverlauf eines Monosignals

c) grobe Aktivitätsstruktur eines Podcasts

d) Wiedererkennung eines Klangereignisses

Video-Inhaltsanalyse

Typische Aufgaben der Video-Inhaltsanalyse sind zum Beispiel:

- Shot-Erkennung, Keyframe-Auswahl, Bewegungsanalyse, Objekterkennung, Aktivitätserkennung

Niveau.

- relativ signalnah: Strukturwechsel / Shots, visuelle und zeitliche Eigenschaften, Bewegungsstruktur
- mittlere Ebene: Objekte, Personen, Szenen
- stark semantisch: Aktivitäten / Handlungen, Zuordnung / Wiedererkennung / Klassifikation

→ Trennung zwischen Merkmalsextraktion und Inferenz

transparente und direkt nachvollziehbaren Bausteine: Histogrammdifferenz, Frame-Differenz / Bewegungsmaß

Video-Inhaltsanalyse

Typische Analyse-Fragen: was soll im Videomaterial erkannt werden

- Wo liegen Strukturwechsel, Shots oder auffällige Abschnitte?
- Welche visuellen und zeitlichen Eigenschaften liegen vor?
- Welche Bewegungsstruktur ist erkennbar?
- Welche Objekte, Personen oder Szenen treten auf?
- Welche Aktivitäten oder Handlungen sind erkennbar?
- Welche Zuordnungen, Wiedererkennungen oder Klassifikationen sind naheliegend?

Beispiele für Analyseziele:
Shot-Erkennung, Keyframe-Auswahl, Bewegungsanalyse, Objekterkennung, Aktivitätserkennung

Video-Inhaltsanalyse

Mit welchen Merkmalen und Verfahren kann man diese Fragen beantworten:

- Strukturwechsel und Shots → z. B. Histogrammdifferenzen, Frame-Vergleiche, Shot-Boundary-Verfahren
- visuelle Eigenschaften → z. B. Helligkeit, Farbe, Histogramme, Kanten, Textur
- Objekte, Personen, Szenen → z. B. Detektions- und Klassifikationsverfahren
- Aktivitäten und Handlungen → z. B. zeitliche Merkmalsfolgen, Video-Classification, Action Recognition
- Zuordnung, Wiedererkennung, Klassifikation → z. B. Merkmalskombinationen, Embeddings, gelernte Repräsentationen

Video-Inhaltsanalyse – harte Schnitte finden

Wenn ein harter Schnitt auftritt, ändert sich das Bild oft sprunghaft. Einfacher Ansatz:

- für jedes Frame die *Farbverteilung* berechnen,
- diese mit der des vorherigen Frames vergleichen,
- große Unterschiede als Hinweis auf einen Schnitt interpretieren

```
hsv = cv2.cvtColor(frame, cv2.COLOR_BGR2HSV)  # von BGR in HSV umwandeln
                # HSV: Hue (Farbton); Saturation (Sättigung); Value (Helligkeit)
hist = cv2.calcHist([hsv], [0, 1], None, [32, 32], [0, 180, 0, 256])
                # Histogramm über Hue und Saturation
                #  = wie häufig kommen bestimmte Farben/Sättigungen vor.
hist = cv2.normalize(hist, hist).flatten()    # Normalisierung, für Vergleich
                # zwischen Frames ;  flatten: macht aus 2D-Matrix ein 1D-Array
score = cv2.compareHist(prev_hist, hist, cv2.HISTCMP_BHATTACHARYYA)
                # Abstandsmaß zwischen 2 Histogrammen (klein → ähnlich, groß → unähnlich)
```

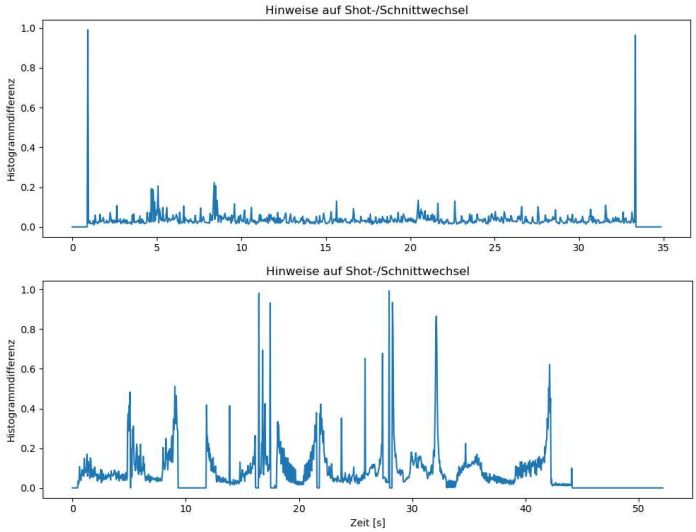
Beispiel 28: Shots / Schnitte finden

```
#cap = cv2.VideoCapture("Ottos_Mops.mp4")
cap = cv2.VideoCapture("Sintel_Trailer.mp4")

prev_hist = None
scores = []
fps = cap.get(cv2.CAP_PROP_FPS) or 25.0

while True:
    ret, frame = cap.read()
    if not ret:
        break
    hsv = cv2.cvtColor(frame, cv2.COLOR_BGR2HSV)
    hist = cv2.calcHist([hsv], [0, 1], None, [32, 32], [0, 180, 0, 256])
    hist = cv2.normalize(hist, hist).flatten()
    if prev_hist is not None:
        score = cv2.compareHist(prev_hist, hist, cv2.HISTCMP_BHATTACHARYYA)
        scores.append(score)
    prev_hist = hist
cap.release()
times = np.arange(len(scores)) / fps
```

Beispiel 28: Shots / Schnitte finden



Ottos Mops.mp4

Sintel Trailer.mp4

Video-Inhaltsanalyse – Bewegung finden

Wenn eine Bewegung auftritt, ändert sich das Bild. Eine einfache Idee ist daher:

- für jedes Bild die Pixelwerte berechnen,
- diese mit denen des vorherigen Frames vergleichen,
- große Unterschiede als Hinweis auf eine Bewegungsaktivität interpretieren

```
prev_gray = cv2.cvtColor(prev, cv2.COLOR_BGR2GRAY)  # von BGR in Graustufen
                                                    #  umwandeln
                                                     # HSV: Hue (Farbton); Saturation (Sättigung); Value (Helligkeit)
gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
                                                    # aktuelles Frame in Graustufen
diff = cv2.absdiff(gray, prev_gray)                 # für jedes Pixel wird der Absolutwert der Differenz
                                                    # zwischen aktuellem und vorherigen Pixel berechnet
motion_scores.append(diff.mean())                   # Mittelwert der Differenzen als
                                                    # Bewegungsmaß
```

Beispiel 29: Bewegung finden

```
cap = cv2.VideoCapture("Sintel_Trailer.mp4")
ret, prev = cap.read()
if not ret:
    raise ValueError("Video konnte nicht gelesen werden.")

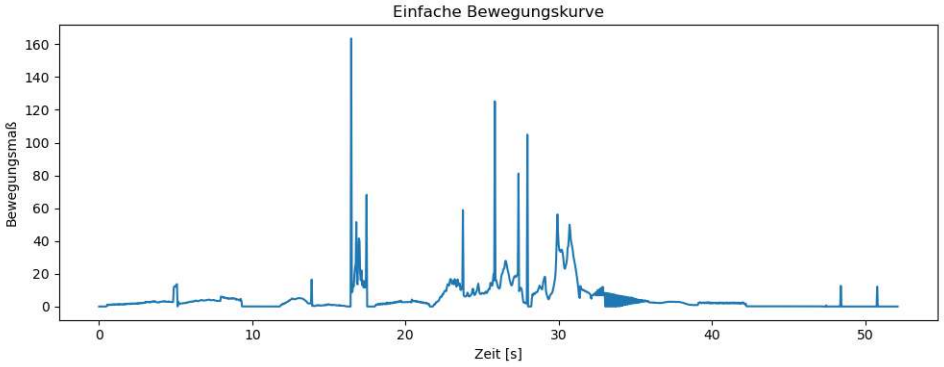
prev_gray = cv2.cvtColor(prev, cv2.COLOR_BGR2GRAY)
fps = cap.get(cv2.CAP_PROP_FPS) or 25.0

motion_scores = []

while True:
    ret, frame = cap.read()
    if not ret:
        break
    gray = cv2.cvtColor(frame, cv2.COLOR_BGR2GRAY)
    diff = cv2.absdiff(gray, prev_gray)
    motion_scores.append(diff.mean())
    prev_gray = gray

cap.release()
```

Beispiel 29: Bewegung finden



Shots finden vs. Bewegung finden (Code 28 und 29)

<u>Fall: Person läuft durch einen Raum</u>

Code 29 reagiert deutlich, weil sich viele Pixel ändern

Code 28 reagiert nur mäßig, weil die Farbverteilung des Bildes ähnlich bleibt.

<u>Fall: Harter Schnitt vom Innenraum zum Strand</u>

Code 28 reagiert deutlich, weil sich die Farbverteilung global stark ändert

Code 28 reagiert auch stark, aber unspezifisch, weil fast alle Pixel anders sind

Code 28 beschreibt eher Änderungen des globalen visuellen Gesamtcharakters eines Frames.
Eignung für Hinweise auf Shot- oder Schnittwechsel.
Code 29 beschreibt eher die direkte Bildänderung zwischen benachbarten Frames.
Eignung als einfaches Maß für Bewegungs- oder Aktivitätsintensität.

Aufgabe 17 – Prüfen Ergebnis Histogrammdifferenz oder Bewegungsanalyse

Bestimmen Sie in einem kurzen Video mit der Histogrammdifferenz oder Bewegungskurve drei Stellen, die Sie als „strukturell auffällig“ markieren würden. Prüfen Sie anschließend im Video:

- liegt ein echter Schnitt vor?
- gibt es hier eine starke Kamerabewegung?
- findet nur Beleuchtungswechsel statt?

Warum reicht ein hohes Bewegungsmaß in einem Video noch nicht aus, um „relevante Aktivität“ zu erkennen? Nennen Sie mindestens drei Gründe.

Aufgabe 18 – ML-Reflexion

Diskutieren Sie zu zweit:

- Was gewinnt man durch ein vortrainiertes Modell?
- Was verliert man gegenüber klassischen Merkmalen?
- Welche Rolle spielen Trainingsdaten und Klassenraum?

ML-gestützte Inhaltsanalyse: Audio und Video

Gründe für ML:

- ✓ komplexe Semantik
- ✓ robustere Klassifikation
- ✓ Transfer Learning

Audio:

- ✓ YAMNet
- ✓ AudioSet
- ✓ Embeddings

Video:

- ✓ Video Classification
- ✓ Action Recognition
- ✓ MoViNet / vortrainierte Modelle

ML-gestützte Inhaltsanalyse vs. klassische Merkmale

Explizit berechnete Merkmale:

- ○
- ○
- ○
- ○

typische Eigenschaften:

- ○
- ○
- ○
- ○

Datengetrieben gelernte Merkmalsrepräsentationen:

- ○
- ○
- ○
- ○

typische Eigenschaften:

- ○
- ○
- ○

ML-gestützte Inhaltsanalyse: Audio und Video
Was gewinnt, was verliert man mit ML?

Diskussionsimpulse:

- ❖interpretierbar vs. leistungsstark
- ❖wenig Daten vs. Trainingsdatensätze
- ❖direkt erklärbar vs. Black-Box-Tendenz
- ❖reproduzierbar vs. domänensensitiv
- ❖geeignet für welchen Anwendungsfall?

ML-gestützte Inhaltsanalyse: Audio und Video

klassische Merkmale:

- transparent
- schnell berechenbar
- begrenzte inhaltliche Aussagekraft

ML-gestützte Verfahren:

- stärkere inhaltliche Zuordnung
- oft robuster bei komplexen Aufgaben
- abhängig von Trainingsdaten und Klassenraum

Beispiele:

- Audio: Audio-Event-Klassifikation, Embeddings
- Video: Objekt-, Szenen- und Aktivitätserkennung

ML-gestützte Audio-Inhaltsanalyse mit YAMNet

YAMNet

- vortrainiertes Audio-Klassifikationsmodell mit 521 Audioereignisklassen
- Grundlage: auf AudioSet basierender Klassenraum
- Eingabe: Mono-Audio mit 16 kHz, normiert auf [-1, 1].
- Ausgabe:
  - Scores pro Klasse und Zeitabschnitt
  - Embeddings als gelernte Repräsentationen
  - internes Spektrogramm

Vereinfachter Ablauf:

➢ Audio auf 16 kHz / Mono vorbereiten

➢ Modell auf die Waveform anwenden

➢ Scores über die Zeit mitteln

➢ Top-Klassen bestimmen

➢ Klassenindizes über die Klassentabelle in Namen übersetzen

https://www.tensorflow.org/hub/tutorials/yamnet

ML-gestützte Audio-Inhaltsanalyse mit YAMNet

YAMNet

- vortrainiertes Audio-Klassifikationsmodell mit 521 Audioereignisklassen
- Grundlage: auf AudioSet basierender Klassenraum
- Eingabe: Mono-Audio mit 16 kHz, normiert auf [-1, 1].
- Ausgabe:
  - Scores pro Klasse und Zeitabschnitt
  - Embeddings als gelernte Repräsentationen
  - internes Spektrogramm

Vereinfachter Ablauf:

➢ Audio auf 16 kHz / Mono vorbereiten

➢ Modell auf die Waveform anwenden

➢ Scores über die Zeit mitteln

➢ Top-Klassen bestimmen

➢ Klassenindizes über die Klassentabelle in Namen übersetzen

https://www.tensorflow.org/hub/tutorials/yamnet

YAMNet is a deep net that predicts 521 audio event classes from the AudioSet-YouTube corpus it was trained on. It employs the Mobilenet_v1 depthwise-separable convolution architecture.

ML-gestützte Audio-Inhaltsanalyse mit YAMNet

➢ Ergebnis des Beispiel-Codes: lesbare TOP 5-Liste.

YAMNet gibt Klassen-Indizes im YAMNet-Klassenraum aus, über `model.class_map_path()` und die CSV-Zuordnung werden daraus Klassennamen.

Der Klassenraum umfasst 521 Audio-Ereignisse.

YAMNet gibt für Mono-WAV mit fs = 16 kHz Scores, Embeddings und ein Spektrogramm zurück; für eine einfache globale Klassifikation mittelt man die Scores über alle Zeitfenster.

`scores`: Wie stark passt jeder Zeitabschnitt zu jeder Klasse?

`mean_scores`: globale Schätzung für den ganzen Audioclip

`top5`: die fünf am höchsten bewerteten Klassen

`class_names[i]`: übersetzt den Klassenindex in einen lesbaren Namen

https://www.tensorflow.org/hub/tutorials/yamnet

Beispiel 30: ML-gestützte Audio-Inhaltsanalyse mit YAMNet

```
import tensorflow as tf
import tensorflow_hub as hub
import numpy as np
from scipy.io import wavfile

model = hub.load("https://tfhub.dev/google/yamnet/1")

sr, wav = wavfile.read("audio_Ottos_Mops.wav")
wav = wav.astype(np.float32)

if wav.ndim == 2:
    wav = wav.mean(axis=1)

wav = wav / np.max(np.abs(wav))

scores, embeddings, spectrogram = model(wav)
mean_scores = tf.reduce_mean(scores, axis=0)

top5 = tf.argsort(mean_scores, direction="DESCENDING")[:5]
print(top5)
```

Beispiel 30: ML-gestützte Audio-Inhaltsanalyse mit YAMNet

Ottos Mops

```
Top-5-Klassen:
0 Speech 0.8272305727005005
494 Silence 0.1117793619632721
3 Narration, monologue 0.023501429706811905
500 Inside, small room 0.010416131466627121
67 Animal 0.007913511246442795
```

Der Audioclip wird sehr deutlich als sprachhaltig erkannt

Es gibt offenbar auch ruhigere bzw. pausenartige Abschnitte

Das Modell ist sich sehr sicher, dass es Sprache ist.
Die speziellere Einordnung als erzählerischer / monologischer Sprachmodus ist nur schwach gestützt.

Raumcharakter / Aufnahmesituation

wahrscheinlich nicht inhaltlich relevant

Achtung: Diese Werte sind keine Wahrscheinlichkeiten!

größter Wert wäre 1.0

Beispiel 30: ML-gestützte Audio-Inhaltsanalyse mit YAMNet

Ottos Mops

```
Top-5-Klassen:
0 Speech 0.8272305727005005
494 Silence 0.1117793619632721
3 Narration, monologue 0.023501429706811905
500 Inside, small room 0.010416131466627121
67 Animal 0.007913511246442795
```

Der Audioclip wird sehr deutlich als sprachhaltig erkannt

Es gibt offenbar auch ruhigere bzw. pausenartige Abschnitte

Das Modell ist sich sehr sicher, dass es Sprache ist.
Die speziellere Einordnung als erzählerischer / monologischer Sprachmodus ist nur schwach gestützt.

Raumcharakter / Aufnahmesituation

wahrscheinlich nicht inhaltlich relevant

Achtung: Diese Werte sind keine Wahrscheinlichkeiten!

Waltz for Rutz

```
Top-5-Klassen:
132 Music 0.8191192746162415
133 Musical instrument 0.1704663187265396
135 Guitar 0.15976472198963165
134 Plucked string instrument 0.15277287364006042
137 Bass guitar 0.06122849881649017
```

größter Wert wäre 1.0

Beispiel 31: ML-gestützte Video-Inhaltsanalyse mit torchvision

```
import torch
import numpy as np
from torchvision.models.detection import (
    fasterrcnn_resnet50_fpn_v2,
    FasterRCNN_ResNet50_FPN_V2_Weights,
)
weights = FasterRCNN_ResNet50_FPN_V2_Weights.DEFAULT
model =  fasterrcnn_resnet50_fpn_v2(weights=weights,box_score_thresh=score_threshold)
model.eval()

categories = weights.meta["categories"]
preprocess = weights.transforms()

device = torch.device("cuda" if torch.cuda.is_available() else "cpu")
model.to(device)
...
   img_tensor = torch.from_numpy(frame_rgb).permute(2, 0, 1)
   batch = [preprocess(img_tensor).to(device)] # Vorverarbeitung passend zum Modell
   prediction = model(batch)[0] # Modell ausführen
```

Beispiel 31: ML-gestützte Video-Inhaltsanalyse mit torchvision





`score_threshold = 0.7`

Beispiel 31: ML-gestützte Video-Inhaltsanalyse mit torchvision






```
score_threshold = 0.5
```

Zusammenfassung

✓Rohdaten sind noch kein Inhalt
✓Merkmale machen ausgewählte Strukturen sichtbar
✓Inferenz verbindet Merkmale mit Deutungen
✓klassische Verfahren sind oft transparenter
✓ML-Verfahren sind oft leistungsfähiger
✓fachliche Bewertung bleibt in beiden Fällen notwendig

Audio- und Video-Inhaltsanalyse folgen derselben Grundlogik:
Rohdaten → Merkmale → Inferenz → Anwendung.

Quellennachweis

Überblick / deutschsprachige Grundlage

Weinzierl, S. (Hrsg.). (2025). *Handbuch der Audiotechnik*. Springer. https://link.springer.com/book/10.1007/978-3-662-60369-7

Audio-Inhaltsanalyse: Werkzeuge und Verfahren

librosa development team. (n.d.). *Feature extraction - librosa documentation.* https://librosa.org/doc/main/feature.html

librosa development team. (n.d.). *librosa.feature.rms - librosa documentation.* https://librosa.org/doc/main/generated/librosa.feature.rms.html

librosa development team. (n.d.). *librosa.feature.spectral_centroid - librosa documentation*. https://librosa.org/doc/main/generated/librosa.feature.spectral_centroid.html

librosa development team. (n.d.). *librosa.yin - librosa documentation*. https://librosa.org/doc/main/generated/librosa.yin.html

librosa development team. (n.d.). *librosa.beat.beat_track - librosa documentation*. https://librosa.org/doc/main/generated/librosa.beat.beat_track.html

Video-Inhaltsanalyse: Werkzeuge und Verfahren

OpenCV. (n.d.). *Getting Started with Videos*. OpenCV Documentation. https://docs.opencv.org/4.x/dd/d43/tutorial_py_video_display.html

OpenCV. (n.d.). *Operations on arrays*. OpenCV Documentation. https://docs.opencv.org/4.x/d2/de8/group__core__array.html

Sai, M. S. P., Chintakindi, S. R., & Sasikala, T. (2024). *Video shot-boundary detection: issues, challenges and solutions*. *Artificial Intelligence Review*. https://link.springer.com/article/10.1007/s10462-024-10742-1

Quellennachweis

ML-gestützte Audio-Inhaltsanalyse

TensorFlow. (n.d.). *Sound classification with YAMNet*. TensorFlow Hub. https://www.tensorflow.org/hub/tutorials/yamnet

TensorFlow Models. (n.d.). *YAMNet README*. GitHub. https://github.com/tensorflow/models/blob/master/research/audioset/yamnet/README.md

Gemmeke, J. F., Ellis, D. P. W., Freedman, D., Jansen, A., Lawrence, W., Moore, R. C., Plakal, M., & Ritter, M. (2017). *Audio Set: An ontology and human-labeled dataset for audio events*. In *Proceedings of the IEEE International Conference on Acoustics, Speech and Signal Processing (ICASSP 2017)*. https://research.google.com/pubs/archive/45857.pdf

Howard, A. G., Zhu, M., Chen, B., Kalenichenko, D., Wang, W., Weyand, T., Andreetto, M., & Adam, H. (2017). *MobileNets: Efficient Convolutional Neural Networks for Mobile Vision Applications*. arXiv. https://arxiv.org/abs/1704.04861

ML-gestützte Video-Inhaltsanalyse

TensorFlow. (n.d.). *MoViNet for streaming action recognition*. TensorFlow Hub. https://www.tensorflow.org/hub/tutorials/movinet

TorchVision contributors. (n.d.). *Models and pre-trained weights*. TorchVision Documentation. https://docs.pytorch.org/vision/main/models.html

Kondratyuk, D., Yuan, L., Li, Y., Zhang, L., Tan, M., Brown, M., & Gong, B. (2021). *MoViNets: Mobile Video Networks for Efficient Video Recognition*. arXiv.

https://arxiv.org/abs/2103.11511